

PLATAFORMA DE TELEMEDICINA · MÓDULO 6 · VERSÃO 1.0

Multi-clínica: Vínculos e Isolamento

A plataforma deixa de ser "um sistema" e passa a ser "um sistema para muitas clínicas". O papel sai da pessoa e vai para o vínculo; toda política de segurança ganha uma segunda pergunta; e os dados que você já criou são levados junto por uma migração escrita à mão.

O que este módulo pressupõe

O Módulo 5 terminou: conta criada pelo site, cinco luzes verdes, commit enviado.

O projeto é telemedicina-modulo-06.zip. Testado do zero: 26 testes automatizados, tipos nos 4 pacotes, next build sem erro, as 6 migrações aplicadas **duas vezes** — num banco vazio e num banco com dados do Módulo 5 — e o isolamento exercitado com duas clínicas, quatro pessoas e 19 verificações de rota de ponta a ponta.

0. O que muda, em uma frase

Antes: "Dra. Ana é médica." Agora: "Dra. Ana é médica **na Clínica A** e médica **na Clínica B**, e o que ela registra numa não existe na outra."

Antes (Módulos 4-5)	Agora (Módulo 6)
perfis.papel dizia quem a pessoa era	vinculos diz quem ela é em cada clínica
papel: paciente, medico, admin	papel: paciente, medico, recepcao, admin_clinica
CPF ficava em pacientes	CPF fica em perfis — é da pessoa, não do papel
RLS perguntava: "esta linha é sua?"	RLS pergunta: "é sua e da clínica onde você tem vínculo?"
/inicio	/c/<clinica>/inicio — a clínica está no endereço
Uma pessoa entrava e pronto	Entra criando uma clínica, ou aceitando um convite

Três tabelas novas — clínicas, vinculos, convites — e três migrações: a 0003 (que leva seus dados junto), a 0004 (que remove o papel antigo com cuidado) e a 0005 (que reescreve as 14 políticas de segurança em 17).

1. Os conceitos

Palavra	O que é (versão de 10 anos)	No nosso projeto
Multi-clínica (multi-tenant)	Um prédio com vários consultórios: mesma portaria, mesma estrutura, mas cada sala tem sua chave e ninguém entra na do vizinho.	Uma instalação, um banco, muitas clínicas isoladas.
Clínica (tenant)	O cliente que contrata. É a fronteira: todo dado de atendimento pertence a uma.	clinicas — CNPJ, endereço no sistema, responsável técnico.
Vínculo	"Fulano é médico aqui." A mesma pessoa pode ter vínculos em vários lugares, com papéis diferentes.	vinculos — pessoa × clínica × papel × status.
Convite	O jeito de entrar numa clínica que não é sua. Vem com papel definido e prazo.	convites — guarda só o hash do código.
Endereço da clínica (slug)	O apelido curto que aparece na URL: /c/clinica-vida/inicio.	Único, escolhido na criação, difícil de mudar depois.

A decisão que molda tudo: isolamento total entre clínicas (ADR-0008)

Entre clínicas diferentes: nada atravessa. Uma médica que atende na Clínica A e na B não vê, logada em A, o que registrou em B. Um paciente atendido nas duas tem histórico separado nas duas.

Dentro da clínica: o contrário — a equipe vê o histórico do paciente ainda que ele passe por vários profissionais. É o que faz um prontuário ser útil, e é como os sistemas de gestão clínica do mercado funcionam.

O motivo não é técnico: a guarda do prontuário é da instituição que o produziu, e compartilhar dado de saúde entre controladores diferentes sem consentimento específico do paciente não se sustenta na LGPD. Compartilhar entre clínicas, se um dia existir, será funcionalidade explícita — nunca o padrão.

2. O modelo novo

```

auth.users (login)
  |
  v
  perfis ..... a PESSOA: nome, CPF, telefone. Uma so, para a vida toda.
    | |
    | +--> medicos      (CRM, UF) ..... da pessoa: vale em qualquer clinica
    | +--> pacientes    (nascimento) ... idem
    |
    +--> vinculos <-- clinicas ..... "sou X na clinica Y"
        ^
        |
        | +--> convites ..... como alguem novo entra
        | +--> consultas ..... todo atendimento tem clinica_id
        +----- auditoria .... todo registro tem clinica_id
  
```

Repare no que ficou **fora** da clínica: nome, CPF e CRM são da pessoa. Isso evita o erro clássico de recadastrar a mesma médica em cada clínica, com CRM digitado de novo (e errado) toda vez. Já a data de nascimento do paciente também é global — é fato biográfico, não registro clínico.

2.1 Os quatro papéis

Papel	O que pode	O que não pode
paciente	Ver o próprio perfil e as próprias consultas naquela clínica; marcar consulta.	Ver outros pacientes; ver a equipe.
medico	Ver os pacientes da clínica e os próprios atendimentos; ver a equipe.	Convidar; ver a auditoria.
recepcao	Ver pacientes e agenda da clínica; ver a equipe.	Convidar; ver a auditoria. (A barreira do prontuário em si vem no módulo do prontuário.)
admin_clinica	Tudo acima, mais: convidar, revogar convite, ler a auditoria da própria clínica.	Ver qualquer coisa de outra clínica — inclusive a auditoria dela.

Uma pessoa pode acumular papéis na mesma clínica (médica e administradora, o caso comum do consultório próprio). Nas telas ela aparece uma vez só, com os papéis somados; nas políticas vale o de maior alcance.

Existe ainda `perfis.suporte_plataforma`: a marca da **nossa** equipe, para tarefas operacionais. Não é papel de vínculo e não aparece em nenhuma política de leitura de dado assistencial. Suporte não lê prontuário — e isso é uma decisão registrada, não um esquecimento.

3. As três migrações (e por que são três)

3.1 A 0003 leva seus dados junto

Você já tem contas criadas no Módulo 5. Uma migração que só criasse tabelas novas quebraria: `consultas.clinica_id` é obrigatório, e as linhas existentes não têm valor nenhum. Então a 0003 foi **gerada** pelo Drizzle e depois **editada à mão** — os blocos marcados com -- [dados] fazem, nesta ordem:

- copiam o CPF de pacientes para `perfis` antes de a coluna antiga sumir;
- criam a **Clínica Demonstração** — mas só se já existir alguém no sistema. Numa instalação nova, nada é criado e a primeira clínica nasce pela tela;
- preenchem `clinica_id` nas consultas e na auditoria existentes;
- transformam cada `perfis.papel` num vínculo com essa clínica (`admin` antigo vira `admin_clinica`);
- dão à clínica uma administração e um responsável técnico — o primeiro médico, ou o primeiro perfil se não houver médico.

Editar migração é errado?

Editar uma migração **gerada e ainda não aplicada** é prática normal — é assim que se faz migração de dados. O que nunca se edita é migração já aplicada em produção (ADR-0004): ali a correção é sempre a próxima migração.

O último item da lista acima só existe porque o pnpm verificar reclamou: a primeira versão criava a clínica **sem administração**, e uma clínica sem admin não consegue nem convidar a equipe. O verificador achou o defeito antes de você.

3.2 A 0004 remove o papel antigo, na ordem certa

As 14 políticas do Módulo 4 e a função `papel_atual()` liam `perfis.papel`. Dropar a coluna com elas de pé simplesmente falha. A 0004 derruba primeiro as políticas e a função, depois a coluna e o tipo.

Entre a 0004 e a 0005, o banco fica com RLS **ligado e sem política nenhuma** nas tabelas antigas — ou seja, ninguém lê nada pela chave pública. É de propósito: se algo interromper a migração no meio, o estado em que se para é o fechado, não o aberto.

3.3 A 0005 reescreve a segurança

Cinco funções novas, todas SECURITY DEFINER com `search_path` vazio, e 17 políticas.

Função	Responde
<code>papel_na_clinica(clinica)</code>	Qual meu papel ali? (NULL se eu não tiver vínculo ativo)
<code>tem_vinculo(clinica)</code>	Eu pertenço a essa clínica?
<code>tem_papel(clinica, papeis[])</code>	Eu sou um desses ali? (usada para "só a equipe", "só admin")
<code>minhas_clinicas()</code>	Quais clínicas são minhas?
<code>compartilha_clinica(pessoa)</code>	Essa pessoa e eu convivemos em alguma clínica?

A última é a que decide se você vê o **nome** de alguém. Sem ela, um paciente enxergaria a lista de todos os perfis do sistema.

4. Três decisões que valem entender

4.1 Por que o banco não guarda "a clínica em que você está"

Seria natural pôr a clínica atual no token (JWT) e as políticas lerem de lá — é rápido, e é o que muito tutorial sugere. Recusamos por três motivos: **claims em metadados do usuário podem ser alterados pelo próprio usuário**, e autorização não se apoia neles; um claim envelhece (suspender um vínculo só teria efeito no próximo login, o que num sistema de saúde é inaceitável); e claim customizado exige configurar um *hook* no painel do Supabase — clique, justamente o que a ADR-0004 proíbe.

Então cada política pergunta à tabela `vinculos`. O custo é uma consulta indexada, resolvido por `vinculos_perfil_clinica` e por envolver cada função em `(SELECT ...)`, o que faz o Postgres avaliar uma vez por consulta em vez de uma vez por linha. Sem esse detalhe, uma tabela grande fica lenta de um jeito difícil de diagnosticar.

4.2 Por que ninguém escreve em `vinculos`

RLS decide **linhas**, não colunas — e `WITH CHECK` não sabe comparar o valor novo com o antigo. Ou seja: uma política que permitisse "editar seu vínculo" permitiria, junto, "editar seu papel" — e qualquer paciente viraria administrador. Por isso `vinculos` não tem política de escrita e tem o privilégio revogado. Promover alguém é rota da API, com auditoria. O mesmo raciocínio aplicamos em `perfis` e `medicos`, onde o `GRANT UPDATE` nomeia as colunas permitidas, uma a uma.

4.3 Por que `/c/clinica/...` e não `clinica.plataforma.com.br`

Subdomínio é o padrão em SaaS B2B e era o plano. Ficou para depois por um motivo concreto: em desenvolvimento, o cookie de sessão gravado em `clinica.localhost` não é visto em `localhost`, o que quebraria o login e a troca de clínica. Como a API recebe a clínica por cabeçalho (`X-Clinica`), migrar para subdomínio depois não muda nada no servidor — só o roteamento do site.

As duas trancas

A API exige o cabeçalho `X-Clinica`, confere o vínculo e responde 403 sem ele.

O banco confere de novo, em cada linha, por conta própria.

Parece redundante e não é: a API usa a chave secreta, que ignora o RLS — então ela precisa se disciplinar sozinha, e o RLS existe exatamente para o dia em que ela falhar. Uma trava que só é testada quando a outra quebra tem de existir antes de quebrar.

5. Atualizar o projeto

Ordem obrigatória

1) `Ctrl+C` no `pnpm dev`. 2) Extrair o zip por cima. 3) `pnpm install` antes de qualquer outro comando.

```
pnpm install
pnpm typecheck
pnpm test
pnpm db:migrar
pnpm verificar
```

O que você deve ver

`pnpm test`: 26 testes (17 da API, 5 do banco, 4 do site).

`pnpm db:migrar`: 3 `migracao(oes)` aplicada(s) e `migracoes` no banco: 6 de 6, com as 8 tabelas listadas.

`pnpm verificar` termina com o passo [9/9]: clínicas cadastradas, toda clínica com administração, toda consulta com clínica, todo vínculo de médico com CRM, convites guardando só o hash.

Confira o que a migração fez com as suas contas — no SQL Editor do Supabase:

```
select p.nome_completo, p.cpf, c.nome_fantasia, v.papel, v.status
from perfis p
left join vinculos v on v.perfil_id = p.id
left join clinicas c on c.id = v.clinica_id
order by 1;
```

Cada conta que você criou no Módulo 5 deve aparecer com um vínculo na **Clínica Demonstração**, com o papel que ela tinha, e o CPF agora em perfis. Se você tinha uma conta de médica, ela também é a administração e a responsável técnica dessa clínica.

6. Provar com duas clínicas

O roteiro abaixo é o coração do módulo: só com **duas** clínicas dá para ver o isolamento acontecendo.

```
pnpm dev
```

6.1 A clínica que veio da migração

1. Abra `http://localhost:3000` e entre com a conta de médica do Módulo 5. Você cai em `/c/demonstracao/inicio` — com uma clínica só, o sistema não faz você escolher.
2. Repare na **barra da clínica**, no topo: nome e seu papel, sempre visíveis. Num sistema onde a mesma pessoa atende em vários lugares, esquecer onde se está é a origem do erro mais caro possível — registrar no lugar errado.
3. Clique em **Equipe**. Você aparece com os papéis somados ("Medico(a), Administracao").

6.2 Convidar alguém

4. Em Equipe, digite um e-mail (pode ser `seu+recepcao@. . .`), escolha **Recepcao** e clique em **Gerar convite**. Aparece um link. **Copie agora**: o sistema guarda só o hash do código, então nem nós conseguimos mostrá-lo de novo. Perdeu? Revogue e gere outro.
5. Abra uma **janela anônima** (Ctrl+Shift+N) e cole o link. Ele manda para o login: crie uma conta nova, complete o perfil, e o convite reaparece. Clique em **Aceitar convite** — você entra como recepção.
6. Nessa conta, digite `/c/demonstracao/equipe` na barra de endereços: **404**. Recepção não administra, e a tela nem admite que existe.

6.3 A segunda clínica, e o isolamento

7. Volte à janela normal (a médica). Vá em **Minhas clínicas > Criar clínica**. Preencha nome, razão social, um CNPJ válido e o endereço (o campo se preenche sozinho a partir do nome). Você cai na clínica nova, como administração.
8. Agora **Minhas clínicas** lista as duas, cada uma com o seu papel. Entre numa e noutra: a lista de médicos muda, a equipe muda. É a mesma pessoa, dois mundos separados.
9. Na janela anônima (a recepção, que só tem vínculo na Demonstração), digite o endereço da clínica nova: `/c//inicio`. **404** — e note que a resposta não é "você não tem acesso", é "não existe". Quem não tem vínculo não precisa nem saber quem é cliente da plataforma.

O que acabou de ser provado

Uma pessoa, várias clínicas, papéis diferentes em cada — sem contas duplicadas.

Quem entra por convite entra com o papel que o convite definiu, e nada além.

Quem não tem vínculo não vê a clínica, não vê a equipe, não vê que ela existe.

E, por baixo de tudo, o RLS repetiria cada uma dessas recusas mesmo que a API tivesse deixado passar.

6.4 O roteiro automático

```
pnpm api:testar-fluxo
```

Com o `pnpm dev` rodando em outro terminal, ele faz sozinho o que você acabou de fazer à mão — e mais: cria dois usuários, cria uma clínica, testa CNPJ inválido, endereço reservado, convite com código inventado, convite usado duas vezes, convite revogado, e tenta furar o RLS por fora da API (inserir um vínculo de administrador para si mesmo, ler clínicas alheias, ler o hash dos convites). No fim apaga tudo o que criou. Termina em "Tudo certo" ou diz em qual passo falhou.

7. O que cada arquivo novo faz

Arquivo	O que faz
<code>packages/db/src/schema/clinicas.ts</code>	O cliente: slug, nome fantasia, razão social, CNPJ, status e responsável técnico (CFM 2.314/2022).

Arquivo	O que faz
packages/db/src/schema/vinculos.ts	Pessoa × clínica × papel × status, com índice único (não há dois papéis iguais na mesma clínica) e o índice que toda política de RLS consulta.
packages/db/src/schema/convites.ts	Convite com hash do código, prazo, quem convidou e quem aceitou.
drizzle/0003_clinicas_e_vinculos.sql	Gerada + editada: cria as três tabelas, move o CPF, dá clínica_id a consultas e auditoria, e leva os dados existentes (blocos [dados]).
drizzle/0004_remove_papel_de_perfis.sql	Derruba as políticas antigas e papel_atual(), depois a coluna e o tipo.
drizzle/0005_rls_multi_clinica.sql	5 funções de contexto, 17 políticas, REVOKE em vínculos, GRANT por coluna em perfis e medicos, comentários nas tabelas.
packages/db/src/verificar-clinicas.ts	pnpm db:verificar-clinicas (passo [9/9]): clínica órfã, consulta sem clínica, médico sem CRM, convite com hash fora do formato.
apps/api/src/contexto.ts	Lê X-Clinica, valida a clínica e o vínculo, entrega req.contexto{clinicalId, slug, papel}. exigirPapel() para as rotas de admin.
apps/api/src/rotas/clinicas.ts	POST /clinicas (com validação de CNPJ e lista de endereços reservados) e GET /clinicas/atual.
apps/api/src/rotas/equipes.ts	GET /membros, GET /medicos, POST /convites, GET /convites, POST /convites/:id/revogar, POST /convites/aceitar.
apps/web/src/lib/clinica.ts	contextoDaClinica(slug): carrega token, perfil e clínica; redireciona quem não tem perfil ou não tem clínica; 404 para clínica que não é sua.
apps/web/src/componentes/BarraDaClinica.tsx	A barra que diz sempre onde você está e com que papel.
apps/web/src/app/clinicas/	Escolher clínica (uma só entra direto) e criar clínica.
apps/web/src/app/c/[clinica]/	Início, equipe (com convites) e diagnóstico, todos dentro da clínica do endereço.
apps/web/src/app/convite/[codigo]/	Aceitar convite, passando por login e perfil se preciso.
docs/adr-0008-multi-clinica.md	As sete decisões deste módulo, com o porquê de cada uma.

8. A regra para tudo que vier depois

1. Tabela assistencial nova nasce com clínica_id NOT NULL e índice por (clínica_id, ...).
2. Na mesma leva, a migração de RLS: ENABLE ROW LEVEL SECURITY e políticas que checam tem_vínculo ou tem_papel. Some as políticas em POLITICAS_ESPERADAS.
3. Rota nova de dentro da clínica usa criarExigirClinica e filtra por req.contexto.clinicaId **além** do RLS — o filtro explícito também deixa o Postgres usar o índice.
4. Auditoria de ação dentro da clínica leva clínicaId. Só o que acontece fora (criar perfil) vai com nulo.

O teste que nunca deve passar

Se alguma tela ou rota conseguir mostrar dado de uma clínica para quem tem vínculo só em outra, é falha grave — não é bug de tela. Antes de entregar qualquer módulo daqui para frente, o roteiro da seção 6.3 é repetido com duas clínicas.

9. Git e checklist


```
git add .  
git commit -m "feat(multi-clinica): clinicas, vinculos, convites e isolamento por RLS"  
git push
```

[] pnpm install; typecheck limpo; pnpm test com 26 passed

[] pnpm db:migrar: 6 de 6; a consulta SQL da seção 5 mostra suas contas com vínculo na Clínica Demonstração

[] pnpm verificar em “Tudo certo”, com [9/9] ok

[] Convite gerado, aceito em janela anônima, e a recepção levou 404 na tela de Equipe

[] Segunda clínica criada; Minhas clínicas lista as duas; quem só tem vínculo numa leva 404 na outra

[] pnpm api:testar-fluxo terminou em “Tudo certo” e limpou o que criou

[] Você sabe explicar por que o papel saiu de perfis e por que ninguém escreve em vínculos

[] Você sabe por que a clínica atual não vive no token

[] Commit feito e enviado, sem nenhum .env

Próximo: Módulo 7 — Agenda e consultas. O médico define seus horários **naquela clínica**, o paciente escolhe um e marca. Entra a regra de não-sobreposição no banco (uma restrição de exclusão do Postgres, que impede duas consultas no mesmo horário do mesmo médico), o status da consulta e a auditoria de cada mudança — tudo já dentro da fronteira que este módulo construiu.